

Referencia Técnica: Librería `fatigueset-lib`

Pipeline de Procesamiento del Dataset `FatigueSet`

24 de marzo de 2026

Índice

1. Introducción	2
1.1. Propósito	2
1.2. Alcance del Dataset	2
1.3. Relación con Series Temporales	2
2. Descripción General de la Arquitectura	3
2.1. Flujo de datos típico	3
3. Módulo <code>loader.py</code>: Carga de Datos	3
3.1. Propósito	3
3.2. Constantes globales	3
3.3. Clase <code>DataLoader</code>	3
3.3.1. Método <code>load_csv(path)</code>	4
3.3.2. Método <code>load_all(dir_path)</code>	4
3.4. Clase <code>FatigueSetLoader</code>	4
3.4.1. Inicialización	4
3.4.2. Métodos de Metadata e Intensidades	4
3.4.3. Métodos de Carga Individual	4
3.4.4. Métodos especializados por sensor	4
3.4.5. Método <code>cargar_todo()</code>	5
3.5. Estructura de archivos soportados	5
4. Módulo <code>processor.py</code>: Procesamiento de Series Temporales	5
4.1. Propósito	5
4.2. Clase <code>DataProcessor</code>	5
4.3. Clase <code>FatigueSetProcessor</code>	5
4.3.1. Inicialización	6
4.3.2. Método <code>calcular_fatigabilidad()</code>	6
4.3.3. Método <code>segmentar_por_fase()</code>	6
4.3.4. Métodos de Extracción de Métricas	6
4.3.5. Método <code>construir_dataset_ml()</code>	7

5. Módulo <code>features.py</code>: Extracción de Características	7
5.1. Propósito	7
5.2. Clase <code>FeatureExtractor</code>	7
5.2.1. Inicialización	7
5.2.2. Métodos de Estadísticas Generales	7
5.2.3. Método <code>features_serie()</code>	7
5.2.4. Método <code>features_eeg_banda()</code>	8
5.2.5. Método <code>features_hrv()</code>	8
5.2.6. Método <code>features_eda()</code>	8
5.2.7. Método <code>features_cognitivas()</code>	8
6. Módulo <code>validators.py</code>: Validación de Integridad	9
6.1. Propósito	9
6.2. Clase <code>DataValidator</code>	9
6.3. Clase <code>FatigueSetValidator</code>	9
6.3.1. Inicialización	9
6.3.2. Método <code>validar_todos()</code>	9
6.3.3. Método <code>resumen()</code>	10
6.3.4. Método <code>imprimir_resumen()</code>	10
7. Módulo <code>utils.py</code>: Funciones Auxiliares	10
7.1. Función <code>convertir_timestamp()</code>	10
8. Flujo de Trabajo Completo	10
8.1. Pipeline End-to-End	10
8.2. Etapas del pipeline	11
9. Integración con Procesamiento de Series Temporales	12
9.1. Sincronización Multi-Sensor	12
9.2. Windowing y Segmentación	12
9.3. Data Leakage Prevention	12
10. Estructura del Dataset ML Agregado	12
10.1. Dimensiones	12
10.2. Columnas principales	13
10.2.1. Metadatos	13
10.2.2. Fatiga Subjetiva (VAS)	13
10.2.3. Señales Fisiológicas	13
10.2.4. EEG	13
10.2.5. Tareas Cognitivas	13
11. Anexos	13
11.1. Tabla de Sensores y Frecuencias	13
11.2. Estructura de Carpetas del Dataset	14
11.3. Referencia de Constantes	14
11.4. Notebooks de Ejemplo	14
11.5. Referencias Cruzadas	14

A. Manejo de Errores Comunes	15
A.1. Error: “No se encontró el dataset”	15
A.2. Error: “Metadata.csv no encontrado”	15
A.3. Advertencia: “Archivo no encontrado en P01/S02”	15

1. Introducción

La librería `fatigueset-lib` es un módulo Python especializado en la carga, validación, procesamiento y extracción de características (*features*) del dataset **FatigueSet**. Este dataset combina múltiples sensores fisiológicos con tareas cognitivas para analizar fatiga física y mental en 12 participantes.

1.1. Propósito

`fatigueset-lib` implementa un pipeline completo y modular que:

1. **Carga** datos desde la estructura de carpetas del dataset `FatigueSet`
2. **Valida** la integridad de los datos (valores nulos, duplicados, tipos)
3. **Procesa** series temporales multimodales (sincronización, segmentación por fases)
4. **Extrae** características estadísticas y especializadas
5. **Agrega** los datos en un dataset listo para Machine Learning (108 filas)

1.2. Alcance del Dataset

- **Participantes:** 12 (IDs: 01–12)
- **Sesiones por participante:** 3 (01, 02, 03)
- **Fases de medición:** 3 por sesión
 - M1: Baseline (*línea base*)
 - M2: Post-ejercicio físico
 - M3: Post-carga cognitiva
- **Dataset agregado final:** $12 \times 3 \times 3 = 108$ filas
- **Sensores:** 6 modalidades (pecho, muñeca, frente EEG, oído, etc.)
- **Tareas cognitivas:** N-Back y Choice Reaction Time (CRT)

1.3. Relación con Series Temporales

Esta librería implementa conceptos clave del documento “*Procesamiento de Datos de Series Temporales*” (véase `../series-temporales/`), incluyendo:

- **Sincronización** de sensores con distintas frecuencias de muestreo
- **Segmentación temporal** en ventanas (fases M1, M2, M3)
- **Normalización** correcta sin data leakage
- **Extracción de features** estadísticas por ventana

☑ Forma correcta (sin leakage)

2. Descripción General de la Arquitectura

La librería está organizada en 5 módulos principales, cada uno responsable de una etapa del pipeline:

Módulo	Clase Principal	Responsabilidad
loader.py	FatigueSetLoader	Carga de datos desde el dataset
processor.py	FatigueSetProcessor	Procesamiento de series temporales
features.py	FeatureExtractor	Extracción de características
validators.py	FatigueSetValidator	Validación de integridad
utils.py	(Funciones)	Funciones auxiliares

2.1. Flujo de datos típico

1. **Carga:** `FatigueSetLoader.cargar_todo()` → Dict con todos los DataFrames
2. **Validación:** `FatigueSetValidator.validar_todos()` → Reporte de integridad
3. **Procesamiento:** `FatigueSetProcessor.construir_dataset_ml()` → Dataset de 108 filas
4. **Features:** `FeatureExtractor.features_*` → Estadísticas especializadas por modalidad

3. Módulo loader.py: Carga de Datos

3.1. Propósito

El módulo `loader.py` proporciona funciones y clases para leer archivos CSV del dataset `FatigueSet` y construir DataFrames manipulables.

3.2. Constantes globales

```
PARTICIPANTES = ['01', '02', ..., '12']
SESIONES = ['01', '02', '03']
FASES_MEDICION = {
    0: 'M1_baseline',
    1: 'M2_post_ejercicio',
    2: 'M3_post_fatiga_mental'
}
INTENSIDAD_NUM = {'low': 1, 'medium': 2, 'high': 3}
CANALES_EEG = ['TP9', 'AF7', 'AF8', 'TP10']
```

3.3. Clase DataLoader

Cargador genérico de archivos CSV.

3.3.1. Método `load_csv(path)`

Lee un archivo CSV y devuelve un `pd.DataFrame`.

```
df = loader.load_csv('datos/archivo.csv')
```

3.3.2. Método `load_all(dir_path)`

Lee todos los archivos `.csv` de un directorio como lista de `DataFrames`.

3.4. Clase `FatigueSetLoader`

Cargador especializado para datos de `FatigueSet`.

3.4.1. Inicialización

```
loader = FatigueSetLoader(  
    base_path='fatigueset', # Ruta del dataset  
    participantes=['01', '02', '03'], # Subconjunto opcional  
    sesiones=['01', '02'] # Subconjunto opcional  
)
```

3.4.2. Métodos de Metadata e Intensidades

- `cargar_metadata()`: Lee `fatigueset/metadata.csv`. Mapeo de participantes a intensidades (low/medium/high).
- `construir_intensidad_map()`: Genera diccionario `{pid: {sesion: intensidad}}`.
- `get_intensidad(participante, sesion)`: Retorna intensidad de una sesión.

3.4.3. Métodos de Carga Individual

- `cargar_csv(participante, sesion, archivo, add_meta=True)`: Carga un archivo específico con metadatos opcionales.
- `cargar_todos(archivo, verbose=True)`: Carga un archivo de todos los participantes y sesiones configurados.

3.4.4. Métodos especializados por sensor

```
df_fatiga = loader.cargar_fatiga() # exp_fatigue.csv  
df_chest = loader.cargar_chest() #  
          chest_physiology_summary.csv  
df_markers = loader.cargar_markers() # exp_markers.csv  
  
df_wrist = loader.cargar_wrist() # Dict con hr, eda, bvp, temp,  
          acc, ibi  
df_eeg = loader.cargar_eeg() # Dict con alpha, beta, delta,  
          gamma, theta  
df_ear = loader.cargar_ear_ppg() # Dict con left, right
```

```
df_nback = loader.cargar_nback()           # exp_nback.csv
df_crt    = loader.cargar_crt()            # exp_crt.csv
```

3.4.5. Método `cargar_todo()`

Carga **todas las señales disponibles** en un único diccionario.

```
dataset = loader.cargar_todo()
# Keys: metadata, fatiga, markers, chest, wrist, eeg, ear, nback,
#       crt, task_switch
```

3.5. Estructura de archivos soportados

El módulo reconoce 43 archivos distribuidos en 6 categorías de sensores:

- **Chest** (7 archivos): ECG, Acc, Respiración, RR intervals
- **Wrist/Empatica E4** (6 archivos): HR, EDA, BVP, Temperatura, Acc, IBI
- **Forehead/Muse** (8 archivos): 5 bandas EEG + Raw + Acc + Gyro
- **Ear/Nokia eSense** (6 archivos): PPG + Acc + Gyro (bilateral)
- **Zephyr** (2 archivos): Activity summary, Device status
- **Experimental** (5 archivos): Fatiga (VAS), Markers, N-Back, CRT, Task-Switch

4. Módulo `processor.py`: Procesamiento de Series Temporales

4.1. Propósito

Procesa datos cargados por `FatigueSetLoader` para extraer agregaciones y métricas por fase.

4.2. Clase `DataProcessor`

Procesador genérico de datos tabulares.

- `process_data(data)`: Suma todas las columnas numéricas.
- `filter_data(data, threshold)`: Filtra filas por umbral.
- `aggregate_data(data)`: Convierte dict a `DataFrame`.

4.3. Clase `FatigueSetProcessor`

Procesador especializado para `FatigueSet`.

4.3.1. Inicialización

```
processor = FatigueSetProcessor(  
    intensidad_map=loader.construir_intensidad_map()  
)
```

4.3.2. Método calcular_fatigabilidad()

Calcula deltas (cambios) de fatiga entre fases.

```
df_fatiga_deltas = processor.calcular_fatigabilidad(df_fatiga)  
# Output: 36 filas (12 participantes × 3 sesiones)  
# Columnas: delta_fisica_ejercicio, delta_mental_ejercicio,  
#           delta_fisica_cognitivo, delta_mental_cognitivo, etc.
```

La salida incluye:

- **Deltas:** M2-M1, M3-M2, M3-M1 (para física y mental)
- **Scores absolutos:** Valores de fatiga en cada fase M1, M2, M3
- **Metadatos:** Participante, sesión, intensidad

4.3.3. Método segmentar_por_fase()

Divide una serie temporal en 3 tercios y retorna el segmento especificado.

```
df_m1 = processor.segmentar_por_fase(df_todo, fase_num=0)  #  
    Primeras 1/3 filas  
df_m2 = processor.segmentar_por_fase(df_todo, fase_num=1)  #  
    Segundas 1/3 filas  
df_m3 = processor.segmentar_por_fase(df_todo, fase_num=2)  #  
    Últimas 1/3 filas
```

4.3.4. Métodos de Extracción de Métricas

Señales fisiológicas:

```
metricas = processor.metricas_segmento(df, ['hr', 'br'], prefijo=  
    'chest_')  
# Output: {'chest_hr_media': ..., 'chest_hr_std': ..., ...}
```

EEG por banda:

```
metricas_alpha = processor.metricas_eeg_segmento(df_eeg, banda='  
    alpha')  
# Output: {'eeg_alpha_media': valor_medio_4_canales}
```

Tareas cognitivas:

```
metricas_nback = processor.metricas_nback(df_nback_fase)  
# Output: {'nback_accuracy': %, 'nback_rt': ms}  
  
metricas_crt = processor.metricas_crt(df_crt_fase)  
# Output: {'crt_rt': ms, 'crt_accuracy': %}
```


4.3.5. Método `construir_dataset_ml()`

Método central que agrega todos los datos en un dataset de 108 filas.

```
df_ml = processor.construir_dataset_ml(  
    df_fatiga=dataset['fatiga'],  
    df_chest=dataset['chest'],  
    df_wrist_eda=dataset['wrist']['eda'],  
    eeg_data=dataset['eeg'],  
    df_nback=dataset['nback'],  
    df_crt=dataset['crt'],  
)  
# Output: 108 filas (12 E 3 sesiones E 3 fases)  
# Columnas: metadatos + features por modalidad
```

La salida estructura cada fila con:

- Identificadores: participante, sesion, fase, intensidad
- Fatiga subjetiva: fatiga_fisica, fatiga_mental
- Features por sensor: media y std de cada modalidad
- Tareas cognitivas: accuracy y RT

5. Módulo `features.py`: Extracción de Características

5.1. Propósito

Extrae características estadísticas y especializadas de series temporales multimodales.

5.2. Clase `FeatureExtractor`

5.2.1. Inicialización

```
extractor = FeatureExtractor(percentiles=[25, 50, 75])
```

5.2.2. Métodos de Estadísticas Generales

- `extract_features(data)`: Media de cada lista en un diccionario.
- `compute_statistics(data)`: Estadísticas (mean, std, min, max) por clave.

5.2.3. Método `features_serie()`

Estadísticas descriptivas de una serie temporal.

```
feats = extractor.features_serie(serie, prefijo='hr_')  
# Output: {  
#     'hr_media': media,  
#     'hr_std': desviación estándar,
```

```
#     'hr_min': mínimo,
#     'hr_max': máximo,
#     'hr_rango': max - min,
#     'hr_cv': coeficiente de variación,
#     'hr_p25': percentil 25, 'hr_p50': mediana, 'hr_p75':
#         percentil 75
# }
```

5.2.4. Método `features_eeg_banda()`

Extrae características EEG por banda y canal.

```
feats = extractor.features_eeg_banda(df_eeg_alpha, banda='alpha')
# Output: features por cada canal (TP9, AF7, AF8, TP10) +
#         promedio global
```

5.2.5. Método `features_hrv()`

Heart Rate Variability (HRV) a partir de intervalos RR.

```
feats = extractor.features_hrv(df_rr_intervals)
# Output: {
#     'hrv_rmssd': raíz cuadrada del promedio de diferencias RRš,
#     'hrv_sdnns': desviación estándar de RR,
#     'hrv_pnn50': % de intervalos RR diferidos > 50ms,
#     'hrv_mean_rr': media de intervalos RR
# }
```

5.2.6. Método `features_eda()`

Electrodermal Activity (EDA) - análisis de respuesta dérmica.

```
feats = extractor.features_eda(df_eda)
# Output: {
#     'eda_media', 'eda_std', 'eda_min', 'eda_max', 'eda_cv':
#         Estadísticas,
#     'eda_pendiente': tendencia lineal (cambio en tiempo),
#     'eda_n_picos_scr': número de picos de respuesta (SCR peaks)
# }
```

Los picos se detectan como cruces por encima de $\mu + \sigma$.

5.2.7. Método `features_cognitivas()`

Características de tareas cognitivas (N-Back y CRT).

```
feats = extractor.features_cognitivas(
    df_nback=df_nback_fase,
    df_crt=df_crt_fase
)
# Output: {
```

```
#      'nback_accuracy': %, 'nback_rt_media': ms, 'nback_rt_std':
      ms,
#      'nback_n_errores': número de errores,
#      'crt_rt_media': ms, 'crt_rt_std': ms
#  }
```

6. Módulo `validators.py`: Validación de Integridad

6.1. Propósito

Valida la integridad del dataset `FatigueSet` (valores ausentes, duplicados, tipos mixtos).

6.2. Clase `DataValidator`

Validador genérico de DataFrames.

- `validate_schema(df, columns)`: Comprueba presencia de todas las columnas.
- `check_missing_values(df)`: Detecta valores NaN.
- `validate_data_quality(df)`: Verifica coherencia de tipos de datos.

6.3. Clase `FatigueSetValidator`

Validador especializado para `FatigueSet`.

6.3.1. Inicialización

```
validator = FatigueSetValidator(
    loader=loader,
    umbral_nulos=5.0 # % máximo permitido
)
```

6.3.2. Método `validar_todos()`

Valida todos los archivos críticos (11 tipos de sensores + 3 tareas).

```
df_validacion = validator.validar_todos()
# Output: DataFrame con una fila por (archivo, participante,
#        sesión)
# Columnas: Archivo, Participante, Sesión, Filas, Nulos (%),
#        Duplicados, Estado
```

Archivos críticos validados:

1. Chest Physiology, ECG Raw
2. Wrist HR, EDA, Temp
3. EEG Alpha, Beta

4. Ear PPG Left
5. Fatigue Scores, N-Back Task, CRT Task

6.3.3. Método `resumen()`

Genera agregaciones de problemas por tipo de archivo.

```
resumen_df, problemas_df = validator.resumen(df_validacion)
# resumen_df: resumen por archivo (sesiones OK, nulos máximos)
# problemas_df: sesiones con estado ''
```

6.3.4. Método `imprimir_resumen()`

Imprime un reporte legible en consola.

7. Módulo `utils.py`: Funciones Auxiliares

7.1. Función `convertir_timestamp()`

Convierte columnas de timestamp (milisegundos desde epoch) a `datetime` de pandas.

```
df = convertir_timestamp(df, col='timestamp')
# Crea columna 'datetime' con formato datetime64
```

8. Flujo de Trabajo Completo

8.1. Pipeline End-to-End

Un uso típico completo:

```
from fatigueset.loader import FatigueSetLoader
from fatigueset.processor import FatigueSetProcessor
from fatigueset.features import FeatureExtractor
from fatigueset.validators import FatigueSetValidator

# 1. CARGA
loader = FatigueSetLoader(base_path='fatigueset')
dataset = loader.cargar_todo()

# 2. VALIDACIÓN
validator = FatigueSetValidator(loader)
validator.imprimir_resumen()

# 3. PROCESAMIENTO
processor = FatigueSetProcessor(
    intensidad_map=loader.construir_intensidad_map()
)
df_ml = processor.construir_dataset_ml(
    df_fatiga=dataset['fatiga'],
```

```

df_chest=dataset['chest'],
df_wrist_eda=dataset['wrist']['eda'],
eeg_data=dataset['eeg'],
df_nback=dataset['nback'],
df_crt=dataset['crt'],
)

# 4. EXTRACCIÓN DE FEATURES AVANZADAS
extractor = FeatureExtractor()
for banda in ['alpha', 'beta', 'theta']:
    df_ml[f'eeg_{banda}_advanced'] = df_ml.apply(
        lambda row: extractor.features_eeg_banda(
            dataset['eeg'][banda], banda
        ), axis=1
    )

# Dataset final: 108 E ~50 columnas
print(df_ml.shape) # (108, 50+)
df_ml.to_csv('dataset_fatigueset_ml.csv', index=False)

```

8.2. Etapas del pipeline

Etapas 1: Carga

- Lectura de 43 archivos CSV distribuidos
- Sincronización de metadatos (intensidades)
- Consolidación en estructura de diccionarios

Etapas 2: Validación

- Comprobación de 11 tipos de sensores
- Detección de valores nulos y duplicados
- Reporte de problemas por sesión

Etapas 3: Procesamiento

- Segmentación temporal en 3 fases por sesión
- Extracción de métricas sumarias (media, std)
- Cálculo de deltas de fatiga (M2-M1, M3-M2, etc.)

Etapas 4: Features

- Features especializadas por modalidad (HRV, EDA, EEG)
- Características de tareas cognitivas (accuracy, RT)
- Preparación de variables para ML

Etapas 5: Agregación

- Dataset final 108 E 50+ columnas
- Listo para modelos predictivos

9. Integración con Procesamiento de Series Temporales

Esta librería implementa en Python los conceptos descritos en el documento “*Procesamiento de Datos de Series Temporales*”.

9.1. Sincronización Multi-Sensor

Problema: Sensores con diferentes frecuencias de muestreo y relojes internos.

Sensor	Frecuencia (Hz)
Chest HR	1
Wrist EDA	4
Forehead EEG	9.13

Solución: `loader.cargar_todo()` sincroniza mediante timestamps en `exp_markers.csv`.

9.2. Windowing y Segmentación

En lugar de ventanas deslizantes explícitas, la librería segmenta cada sesión en 3 fases por tercio temporal:

$$\text{fila} = \lfloor \frac{\text{fase_num} \times n}{n_fases} \rfloor, \quad \text{fila} = \lfloor \frac{(\text{fase_num} + 1) \times n}{n_fases} \rfloor$$

Método: `FatigueSetProcessor.segmentar_por_fase()`

9.3. Data Leakage Prevention

La librería implementa normalización sin contaminación:

1. Separación temporal: Split determinístico por fases
2. Normalización independiente por segmento si es necesario
3. Metadatos claros (participante, sesión, fase)

10. Estructura del Dataset ML Agregado

10.1. Dimensiones

- **Filas:** 108 (12 participantes $\times$ 3 sesiones $\times$ 3 fases)
- **Columnas:** Aproximadamente 50+, distribuidas como:
 - 6 columnas de metadatos
 - 15-20 features fisiológicas
 - 10-15 features EEG
 - 8-10 features cognitivas
 - 5-10 features especializadas (HRV, EDA)

10.2. Columnas principales

10.2.1. Metadatos

```
participante, sesion, fase, intensidad, intensidad_num, fase_num
```

10.2.2. Fatiga Subjetiva (VAS)

```
fatiga_fisica, fatiga_mental
```

10.2.3. Señales Fisiológicas

```
chest_hr_media, chest_hr_std, chest_br_media, chest_br_std,  
  chest_hrv_media  
eda_media, eda_std, eda_pendiente, eda_n_picos_scr
```

10.2.4. EEG

```
eeg_alpha_media, eeg_beta_media, eeg_theta_media  
eeg_alpha_TP9_media, eeg_alpha_AF7_media, ... (por canal)
```

10.2.5. Tareas Cognitivas

```
nback_accuracy, nback_rt_media, nback_rt_std, nback_n_errores  
crt_rt_media, crt_rt_std
```

11. Anexos

11.1. Tabla de Sensores y Frecuencias

Dispositivo	Sensor	Frecuencia (Hz)
3*Zephyr BioHarness	Heart Rate	1
	Breathing Rate	1
	Corrected GSR	1
6*Empatica E4	HR (IBI)	1
	EDA	4
	BVP	64
	Temperatura piel	4
	Acelerómetro	32
5*InteraXon Muse	EEG TP9, AF7, AF8, TP10	9.13
	5 bandas (alpha, beta, delta, gamma, theta)	1
	PPG (Ear channels)	64

11.2. Estructura de Carpetas del Dataset

```
fatigueset/
01/
  01/ (sesión 1)
    chest_*.csv
    wrist_*.csv
    forehead_eeg_*.csv
    exp_fatigue.csv
    exp_nback.csv
    exp_crt.csv
  02/ (sesión 2)
  03/ (sesión 3)
02/ ... 12/
metadata.csv (mapeo participante intensidades)
pre_task_survey.xlsx
preliminary_questionnaire.xlsx
```

11.3. Referencia de Constantes

```
# Mapa de fases
FASES_MEDICION = {
    0: 'M1_baseline',
    1: 'M2_post_ejercicio',
    2: 'M3_post_fatiga_mental'
}

# Mapa de intensidades
INTENSIDAD_NUM = {'low': 1, 'medium': 2, 'high': 3}

# Canales EEG del Muse
CANALES_EEG = ['TP9', 'AF7', 'AF8', 'TP10']
```

11.4. Notebooks de Ejemplo

La librería incluye ejemplos prácticos:

- `notebooks/example_usage.ipynb`: Demostración completa del pipeline
- `../Jupyterers/FatigueSet Procesado de datos.ipynb`: Pipeline original del proyecto
- `../Jupyterers/windowing_data_leakage.ipynb`: Análisis de data leakage
- `../Jupyterers/sincronizacion_normalizacion.ipynb`: Sincronización multi-sensor

11.5. Referencias Cruzadas

Para más detalles sobre conceptos de procesamiento de series temporales:

- ../series-temporales/procesamiento_datos_series_temporales.pdf
- Secciones: Sincronización, Windowing, Data Leakage, Normalización

A. Manejo de Errores Comunes

A.1. Error: “No se encontró el dataset”

```
FileNotFoundError: No se encontró el dataset en: fatigueset
```

Solución: Asegurar que la ruta `base_path` es correcta y que la carpeta contiene las subcarpetas 01–12.

A.2. Error: “Metadata.csv no encontrado”

Solución: Verificar que existe `fatigueset/metadata.csv`.

A.3. Advertencia: “Archivo no encontrado en P01/S02”

Esto es esperado si no todos los archivos están disponibles para todas las sesiones. La librería continúa procesando.